Windowed Hankel KEM: A Compact Lattice-Based KEM from Truncated Linear Convolution

Marcos Mendoza III
Independent Researcher

October 2026

Abstract

We present *Windowed Hankel KEM*, a lattice-based key encapsulation mechanism whose public key is a noisy *middle product*: the coefficients of the linear convolution of a long pseudorandom sequence with a short secret that depend on every secret coordinate. The public matrix is thus a Hankel matrix described by a single sequence, which places the scheme between unstructured LWE and ring- or module-LWE. Middle-Product LWE is built on the same operation, but our instance makes the long operand public rather than secret, so the known reductions do not apply. We state the resulting problem precisely and study it through its structure and through concrete cryptanalysis. With $n = 640$, $q = 2^{12}$ and binomial noise, the scheme encapsulates a 256-bit message with a 1,196-byte public key and a 1,356-byte ciphertext, and its decryption failure rate is below $2^{-134.1}$. If the Hankel structure gives an attacker no advantage, the cheapest generic attack found by the lattice estimator costs $2^{137.1}$ in the Core-SVP model, against $2^{115.5}$ for Kyber-512 under the same model. That assumption is the crux. We test it against the most natural structural attack, through an embedding into the cyclic ring $\mathbb{Z}_q[X]/(X^{1407} - 1)$, and find no advantage, but we offer the scheme primarily as a precise problem statement for cryptanalysis rather than as a scheme ready for use. A constant-time C implementation completes a handshake in 51 μs on an Apple M3 Max.

Keywords: post-quantum cryptography, key encapsulation, learning with errors, middle-product LWE, Hankel matrices.

1 Introduction

Lattice-based key encapsulation mechanisms (KEMs) trade algebraic structure for efficiency. Unstructured LWE [25], as used by FrodoKEM [8], rests on worst-case reductions from standard lattice problems but needs about 19 KB per exchange at the 128-bit level. Module-LWE, as used by ML-KEM [9, 20], works in the ring $\mathbb{Z}_q[X]/(X^n + 1)$ and needs about 1.6 KB, at the price of a rich algebraic structure whose security has been studied extensively [22].

This paper studies an intermediate point. The public matrix is a *Hankel* matrix $H_{i,j} = a_{i+j}$ generated by a single pseudorandom sequence a, so that multiplying by it computes a contiguous slice of the coefficients of a polynomial product in $\mathbb{Z}_q[X]$, with no reduction modulo any polynomial. Which slice is published matters. The outermost coefficients of a linear convolution involve only a few secret coordinates, and publishing them would expose the secret one coordinate at a time. We therefore publish only the *fully supported* coefficients, each of which involves all n secret coordinates. This is the *middle product* of Roşca, Sakzad, Steinfeld and Stehlé [26].

Their Middle-Product LWE problem (MP-LWE) uses the same operation, but places the long operand on the secret side and draws many short public operands. That arrangement is what makes reductions from Ring-LWE possible, and it is also a large part of why MP-LWE schemes are big: the Titanium KEM [27, 28] has a 16 KB public key. We invert the arrangement, publishing one long sequence and keeping the secret short, which brings a complete exchange

down to 2,528 bytes. In exchange, we have no reduction. The security of the scheme rests on a new assumption, the hardness of the *Windowed Hankel LWE* problem, which we support with a precise comparison to MP-LWE, an analysis of how known structure-exploiting attacks fare against it, and concrete estimates for every generic attack family.

Contributions.

1. A KEM built on windowed Hankel products, in which a 128-coordinate window carries a 256-bit message by 4-PAM modulation and the modulus $q = 2^{12}$ reduces all arithmetic to masking (Section 5).

2. A definition of the underlying problem and an analysis of its structure: how it differs from MP-LWE, why published structure-exploiting attacks do not apply, an embedding into a cyclic ring, and the effect of the power-of-two modulus (Section 4).

3. A parameter study, concrete estimates for all generic attack families, and an exact Chernoff bound on the decryption failure rate (Sections 6 and 7), with candidate parameters for NIST Categories 3 and 5 (Appendix D).

4. A constant-time C implementation that is bit-exact with the reference implementation, with a static audit of its compiled code and statistical timing tests (Section 8).

After reviewing related work (Section 2) and fixing notation (Section 3), we define the problem (Section 4), specify the scheme and its parameters (Sections 5 and 6), analyze its security (Section 7) and describe the implementation (Section 8). Section 9 closes with limitations and open questions.

2 Related Work

Structured LWE. Ring-, Module- and Polynomial-LWE gain efficiency by working in polynomial rings [22]. Peikert and Pepin cast all of these, together with Order-LWE and MP-LWE, as instances of a single generalized LWE problem defined by a bilinear form, and give simpler and tighter reductions among them [23]. In their framework, the matrix slices of the MP-LWE bilinear form are exactly the standard basis of the Hankel matrices [23, Sect. 5], which is the precise sense in which our scheme is Hankel-structured.

Middle-Product LWE. Roşca et al. introduced MP-LWE and reduced Polynomial-LWE to it for every defining polynomial in a large family, so that MP-LWE is at least as hard as the hardest of these Polynomial-LWE instances [26]. Steinfeld, Sakzad and Zhao tightened the reduction and used it to set the parameters of Titanium [28], a first-round candidate in the NIST process that did not advance to the second round [1]. Peikert and Pepin reduce Ring-LWE to MP-LWE through Order-LWE with less error growth [23], and three recent works widen the coverage further: to at least 90% of bounded-coefficient polynomials of a given degree, experimentally [21]; to all number fields defined by monic irreducible polynomials with polynomially bounded coefficients [24]; and with less noise growth for trace-based reductions [10]. Bai et al. prove hardness for secrets that are small relative to the modulus, which their signature scheme MPSign requires [5]. Yang and Huang generalize the middle product to a universal product with more flexible parameters [30]. MP-LWE also supports identity-based encryption [16]; Tezuka and Tanaka survey these applications [29].

Cryptanalysis. Three published attacks exploit structure of the kind considered here: a dimension reduction for MP-LWE by Steinfeld et al. [28], a coefficient-by-coefficient recovery of small secrets by Bai et al. [5], and evaluation attacks on Ring-LWE over rings whose defining polynomial has a root of small order modulo q, by Elias, Lauter, Ozman and Stange [12].

Section 4.3 explains why none of them applies directly. Compact-LWE, a lightweight LWE variant broken shortly after publication [7], is a reminder that compact variants deserve suspicion. For concrete estimates we follow the Core-SVP methodology [4] and the analysis of Albrecht, Player and Scott [3], as implemented in the lattice estimator [2], and also report the cost model of MATZOV [17].

We found no published analysis of the setting studied here, with one long public sequence, a short binomial secret and narrow noise.

3 Preliminaries

Throughout, $q = 4096$ and $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$, with elements represented in $[0, q)$, and vectors are indexed from 0. SHAKE256(x, ℓ) denotes ℓ bytes of SHAKE256 output on input x [19], SHA256 denotes SHA-256 [18], and $\|$ denotes concatenation. The centered binomial distribution CBD_η outputs $\sum_{i<\eta}(x_i - y_i)$ for independent uniform bits x_i, y_i; in particular, CBD_2 takes the values $-2, \ldots, 2$ with probabilities $(1, 4, 6, 4, 1)/16$ and has variance 1. The scheme samples one CBD_2 value from the low four bits of each byte of SHAKE256 output.

3.1 Hankel matrices and the middle product

Definition 1 (Hankel matrix)**.** For $a = (a_0, \ldots, a_{m+n-2}) \in \mathbb{Z}_q^{m+n-1}$, the $m \times n$ *Hankel matrix* of a is $H_m(a) \in \mathbb{Z}_q^{m\times n}$ with $H_m(a)_{i,j} = a_{i+j}$.

Write $a(X) = \sum_t a_t X^t$, and let $\tilde{s}(X) = \sum_{j<n} s_j X^{n-1-j}$ be the reversal of $s \in \mathbb{Z}_q^n$. Then

$$(H_m(a)\, s)_i = \sum_{j=0}^{n-1} a_{i+j}\, s_j = [X^{i+n-1}]\; a(X)\, \tilde{s}(X).$$

Of the $m + 2n - 2$ coefficients of $a(X)\tilde{s}(X)$, the vector $H_m(a)\, s$ keeps the m in the middle, which are exactly those involving all n entries of s; this is the *m-middle product* of [26]. Unlike circulant or negacyclic matrices, Hankel matrices are not closed under multiplication and do not form a ring.

Because convolution is commutative, we may also write $H_m(a)\, s = T_m(s)\, a$, where $T_m(s) \in \mathbb{Z}_q^{m\times(m+n-1)}$ is the banded matrix with $T_m(s)_{i,t} = s_{t-i}$ for $0 \le t - i < n$ and 0 otherwise. The two forms correspond to the two ways of assigning the long and the short operand, which is the distinction that matters in Section 4.2.

3.2 4-PAM modulation

A 256-bit message is carried by $W = 128$ coordinates, two bits each. Symbol $\mu \in \{0, 1, 2, 3\}$ is encoded as $\mathrm{PAM}(\mu) = \mu \cdot q/4 = 1024\mu$, and symbol w of the message consists of bits $2(w \bmod 4)$ and $2(w \bmod 4) + 1$ of byte $\lfloor w/4 \rfloor$.

Proposition 1 (Decoder)**.** If $v = 1024\mu + e \bmod 4096$ with $-512 \le e \le 511$, then $((v + 512) \bmod 4096) \gg 10 = \mu$.

Proof. Since $v + 512 \equiv 1024\mu + (e + 512) \pmod{4096}$ with $0 \le e + 512 \le 1023$, we have $\lfloor((v + 512) \bmod 4096)/1024\rfloor = \mu$, and division by 1024 is a right shift by 10. □

3.3 KEMs and the Fujisaki–Okamoto transform

A KEM is secure if the shared key is indistinguishable from random even to an adversary with a decapsulation oracle (IND-CCA). The Fujisaki–Okamoto transform with implicit rejection builds such a KEM from a public-key encryption scheme that is IND-CPA secure and δ-correct, where δ

is the probability that decryption of an honestly generated ciphertext fails. Hofheinz, Hövelmanns and Kiltz analyze it in the random oracle model [13], and Jiang et al. [14] and Bindel et al. [6] in the quantum random oracle model; in all of these bounds the adversary's advantage grows with δ.

4 The Windowed Hankel LWE Problem

4.1 Definition

Definition 2 (Windowed Hankel LWE)**.** Let $n, k \geq 1$, $m_1, \ldots, m_k \geq 1$ and $q \geq 2$, and let χ_s and χ_e be distributions on $\mathbb{Z}$. In $\mathsf{WH\text{-}LWE}_{n,(m_1,\ldots,m_k),q,\chi_s,\chi_e}$, a secret $s \leftarrow \chi_s^n$ is sampled, and for each $\ell \leq k$ independently a uniform $a^{(\ell)} \in \mathbb{Z}_q^{n+m_\ell-1}$ and an error $e^{(\ell)} \leftarrow \chi_e^{m_\ell}$, giving $b^{(\ell)} = H_{m_\ell}(a^{(\ell)})\, s + e^{(\ell)} \bmod q$. The *search* problem is to find s from $(a^{(\ell)}, b^{(\ell)})_{\ell \leq k}$; the *decision* problem is to distinguish these pairs from pairs in which each $b^{(\ell)}$ is replaced by a uniform vector.

The scheme relies on two instances, both with $n = 640$, $q = 4096$ and $\chi_s = \chi_e = \mathrm{CBD}_2$: the public key is an instance with $k = 1$ and $m_1 = 768$, and the ciphertext, once the public key has been replaced by a uniform vector in the security argument, is an instance with $k = 2$ and $(m_1, m_2) = (768, 128)$ whose secret is the ephemeral r (Section 5.3). Read as plain LWE, each is an instance with secret dimension n and $m_1 + \cdots + m_k$ samples whose matrix happens to be Hankel.

4.2 Comparison with Middle-Product LWE

In MP-LWE [26, Def. 3.5], the secret is a uniform polynomial of degree less than $n + d - 1$, and the adversary receives arbitrarily many samples, each pairing a fresh uniform polynomial a_i of degree less than n with d noisy coefficients of the middle product of a_i and the secret. Roşca et al. observe that this is LWE with correlated samples, sample i multiplying the secret by a banded $d \times (n + d - 1)$ Toeplitz matrix built from a_i [26, Sect. 3.2]. In the language of Peikert and Pepin, sample i applies a $d \times n$ Hankel matrix generated by the *secret* to the coefficient vector of a_i [23, Def. 5.1]. Our problem is the mirror image: the Hankel matrix is generated by the *public* sequence and applied to the secret. Table 1 sets the two side by side.

	MP-LWE [23, 26]	Windowed Hankel LWE (this work)
Hankel matrix generated by	the secret (length $n + d - 1$)	the public sequence (length $n+m-1$)
Applied to	a fresh short public polynomial per sample	the short secret
Number of samples	arbitrarily many; $\Theta(\log n)$ in the encryption scheme of [26]	one (public key) or two (ciphertext)
Secret	uniform, or Gaussian of width growing with $\sqrt{n}$ [5]	CBD_2, standard deviation 1
Error	Gaussian of width $\Omega(\sqrt{n})$	CBD_2, standard deviation 1
Hardness	reductions from Ring-/Polynomial-LWE	none known

Table 1: MP-LWE and the problem underlying Windowed Hankel KEM.

The known reductions depend on the MP-LWE arrangement at every step. They embed a Polynomial-LWE secret into the long MP-LWE secret, rerandomize across many samples, and need noise wide enough to absorb the distortion of the embedding [21, 23, 24, 26]; Roşca et al. also choose the width $\Omega(\sqrt{n})$ explicitly to rule out attacks in the style of Arora and Ge [26, Sect. 4]. With a single public sequence, a short secret and noise of standard deviation 1, none of these

steps is available to us. Commutativity does not repair this: $H_m(a)\,s = T_m(s)\,a$ is an identity between matrices, not between security games, since the noise, the entropy and the choice of hidden operand all sit on different sides.

What can be said is limited. Windowed Hankel LWE is plain LWE whose matrix happens to be Hankel, so any algorithm for LWE with secret dimension 640 and 768 samples solves it; it is *at most* as hard as plain LWE, which is why generic estimates are the optimistic direction. The converse fails for a dimension count: a Hankel matrix has $n + m - 1 = 1{,}407$ free entries against $nm = 491{,}520$ for a uniform matrix, so a generic LWE instance cannot be embedded in one. Nor is the public key statistically hidden, since the secret carries about 1,300 bits of entropy while b has $768 \times 12 = 9{,}216$ bits; hiding is computational and depends on the structure. Finally, the cyclic restatement of Section 4.3 does not import hardness from cyclotomic Ring-LWE: the ring $\mathbb{Z}_q[X]/(X^L - 1)$ has the small-order root 1, and relating it to a cyclotomic ring would mean working with complete samples, which is exactly what the withheld coefficients prevent. Windowed Hankel LWE thus lies off both branches of the structured-LWE landscape, neither unstructured nor inherited from a ring, and we treat its hardness as an assumption in its own right, checking resistance to Arora–Ge attacks concretely in Section 7.

4.3 Structure and known attacks

A Hankel matrix $H_m(a)$ is determined by $n + m - 1$ values rather than nm, and its rows are shifts of one another. We now ask whether known attacks can exploit this; Section 7 treats the generic attacks.

Full support defeats truncation and peeling. Since $b_i = \sum_{j<n} a_{i+j} s_j + e_i$, every published coordinate depends on all n secret coordinates: the structure of $H_m(a)$ forces none of its entries to be zero. This rules out two published techniques. Steinfeld et al. reduce the dimension of the primal attack on MP-LWE by discarding output coordinates, which removes secret coordinates because each output sees only a window of the long secret [28, Prop. 2]. Here, discarding outputs removes no secret coordinate; it merely selects a subset of samples, which the generic attacks already optimize. Bai et al. recover a small secret coefficient by coefficient from the lowest coefficients of a full, noiseless product, where the r-th coefficient involves only secret coefficients of index at most r [5, Sect. 4]. Our window excludes exactly those coefficients, and our outputs are noisy.

Embedding into a cyclic ring. Let $L = n + m - 1$ be the length of the public sequence. Reducing $a(X)\tilde{s}(X)$, of degree at most $L + n - 2$, modulo $X^L - 1$ folds only the coefficients of degree at least L, onto degrees $0, \ldots, n - 2$. The published coefficients, of degrees $n - 1$ to $L - 1$, are untouched, so

$$b_i = [X^{i+n-1}]\,\big(a(X)\,\tilde{s}(X) \bmod (X^L - 1)\big) + e_i, \qquad 0 \le i < m.$$

A public key is therefore a Ring-LWE-type sample over $\mathbb{Z}_q[X]/(X^L - 1)$ whose secret is supported on n of the L coefficients and of which only m output coefficients are revealed. This ring is far from the cyclotomic rings recommended for Ring-LWE, since $X^L - 1$ has the root 1 modulo every q. Over such rings, evaluating a sample at a root α of the defining polynomial collapses it to a one-dimensional sample $b(\alpha) = a(\alpha)s(\alpha) + e(\alpha)$, and this is the basis of the attacks of Elias et al. [12].

The evaluation, however, needs all L coefficients of the sample, and $n - 1 = 639$ of them are withheld. With only the window, an attacker can apply a linear functional $\varphi \in \mathbb{Z}^m$ and obtain

$$\varphi^\top b = \big(H_m(a)^\top \varphi\big)^\top s + \varphi^\top e.$$

For a factor g of $X^L - 1$ over $\mathbb{Z}_q$ of degree k, let $R_g \in \mathbb{Z}_q^{k \times n}$ be the matrix of the map $s \mapsto \tilde{s} \bmod g$. Functionals with $H_m(a)^\top \varphi = R_g^\top c$ for some $c \in \mathbb{Z}_q^k$ give $\varphi^\top b = c^\top(\tilde{s} \bmod g) + \varphi^\top e$, which depends on s only through the k coefficients of $\tilde{s} \bmod g$; for $g = X - 1$ this is $s(1) = \sum_j s_j$. The all-ones functional does not qualify, because the entries $\sum_{i<m} a_{i+j}$ of $H_m(a)^\top \mathbf{1}$ vary with j: truncation does not commute with evaluation. The qualifying functionals form a q-ary lattice Λ_g of rank m, cut out by $n - k$ linear constraints modulo q. To use them, an attacker needs vectors of Λ_g short enough that $\varphi^\top e$ is far from uniform, and must then guess $\tilde{s} \bmod g$.

This is the situation of the dual hybrid attack of Section 7, which likewise drops $n - k$ constraints and guesses k quantities, except that there the guessed quantities are k coordinates of s. Since $1407 = 3 \cdot 7 \cdot 67$ and 2 has order 2, 3, 6 and 66 modulo 3, 7, 21 and 67, the polynomial $X^{1407} - 1$ factors over $\mathbb{Z}_q$ into irreducible factors of degrees 1, 2, 3, 3, 6 and 6 and twenty-one factors of degree 66. Appendix C reports two computations for the low-degree factors. At full size, Λ_g has determinant exactly q^{n-k} for every random sequence tested, the same as the corresponding dual-hybrid lattice, while the quantity to be guessed carries more entropy than k coordinates of s: 6.7 bits for $s(1)$ against 2.0 bits for one coordinate, and 36.0 against 6.1 bits for each cubic factor. At reduced dimensions, where shortest vectors can be computed exactly, the lattices Λ_g built from Hankel matrices have shortest vectors within a few percent of the Gaussian heuristic, like lattices built from uniform matrices: over 315 instances of each kind, the mean ratio lies between 0.99 and 1.02 for every factor and size. Functionals of this kind therefore give an attacker nothing that the generic dual hybrid attack does not already provide more cheaply. This does not exclude other uses of the ring structure, which remain open.

Power-of-two modulus. Reducing a sample modulo 2^k, for $k < 12$, gives an LWE instance with the same secret and error modulo 2^k. For small k such an instance does not even determine s. Modulo 2, the error is uniform, since odd values of CBD_2 have total probability $1/2$, so b mod 2 is independent of s. Modulo 4 and 8, the error has entropy 1.906 and 2.031 bits, so each of the 768 samples carries at most 0.094 and 0.969 bits of information about s mod 2^k: at most 72 and 744 bits in total, against an entropy of 1,220 and 1,300 bits for s mod 4 and s mod 8. For larger k, the reduced instance has a smaller ratio of modulus to noise than the original, which makes lattice attacks harder, not easier.

Reuse of the public sequence. The sequence a generates both the public key and the first component of every ciphertext, much as ML-KEM reuses its matrix. Each key pair has its own seed, so no structure is shared between users.

5 The Scheme

The parameters are $n = 640$, $W = 128$, $m = n + W = 768$, $q = 4096$ and $\eta = 2$; the public sequence has length $L = 2n + W - 1 = 1407$, and $H = H_m(a)$.

5.1 Public-key encryption

Algorithm 1 CPA.KeyGen(σ)

Require: $\sigma \in \{0,1\}^{256}$

1: $(seed_a \,\|\, seed_e) \leftarrow \mathrm{SHAKE256}(\sigma \,\|\, \texttt{"_hankel_cpa_keygen_seed"}, 64)$
2: $a \leftarrow (\mathrm{SHAKE256}(seed_a, 2L)$ read as little-endian 16-bit words$) \bmod q$
3: $\beta \leftarrow \mathrm{SHAKE256}(seed_e \,\|\, \texttt{"_hankel_noise"}, n + m)$
4: $s \leftarrow \mathrm{CBD}_2(\beta[0{:}n])$; $e \leftarrow \mathrm{CBD}_2(\beta[n{:}n+m])$
5: $b \leftarrow Hs + e \bmod q$
6: **return** $pk = (seed_a, b)$, $sk = s$

Algorithm 2 CPA.Enc$(pk, M; c)$

Require: $pk = (seed_a, b)$, message $M \in \{0,1\}^{256}$, coins $c \in \{0,1\}^{256}$

Regenerate a from $seed_a$
$\gamma \leftarrow \mathrm{SHAKE256}(c \,\|\, \texttt{"_hankel_encaps_coins"}, n+m+W)$
$r \leftarrow \mathrm{CBD}_2(\gamma[0{:}n]);\;\; e_1 \leftarrow \mathrm{CBD}_2(\gamma[n{:}n+m]);\;\; e_2 \leftarrow \mathrm{CBD}_2(\gamma[n+m{:}n+m+W])$
$u \leftarrow Hr + e_1 \bmod q$
$K_w \leftarrow \sum_{j=0}^{n-1} r_j\, b_{w+j} \bmod q \;\;$ for $0 \le w < W$ ▷ $K = H_W(b)\, r$
$v \leftarrow K + \mathrm{PAM}(M) + e_2 \bmod q$
return (u, v)

For compatibility with the reference implementation, key generation also sets $s_0 = 1$ if $s = 0$, without branching. The event has probability $2^{-905.6}$ and has no effect on the analysis.

Algorithm 3 CPA.Dec$(s, (u, v))$

$K'_w \leftarrow \sum_{k=0}^{n-1} s_k\, u_{w+k} \bmod q \;\;$ for $0 \le w < W$ ▷ $K' = H_W(u)\, s$
return Decode$(v - K' \bmod q)$ ▷ Proposition 1, coordinate-wise

Correctness. Substituting $b = Hs + e$ and $u = Hr + e_1$ gives

$$K_w = \sum_{j,k} a_{w+j+k}\, r_j s_k + \sum_j r_j e_{w+j}, \qquad K'_w = \sum_{j,k} a_{w+j+k}\, r_j s_k + \sum_k s_k e_{1,w+k}.$$

The double sums agree because each term depends only on $j + k$; this is the Hankel counterpart of the identity $r^\top(As) = (r^\top A)s$ behind LWE-based key exchange [15]. Hence

$$v_w - K'_w = \mathrm{PAM}(M)_w + \underbrace{\sum_{j=0}^{n-1} r_j e_{w+j} - \sum_{k=0}^{n-1} s_k e_{1,w+k} + e_{2,w}}_{e_{\mathrm{dec},w}} \pmod q,$$

and by Proposition 1 decryption succeeds whenever $-512 \le e_{\mathrm{dec},w} \le 511$ for every w. The term e_2 cannot be omitted: without it, $v - \mathrm{PAM}(M) = H_W(b)\, r$ would hand an attacker W noiseless linear equations in the ephemeral secret r.

5.2 The KEM

The KEM applies the Fujisaki–Okamoto transform with implicit rejection [13]. The secret key holds s as signed bytes, a rejection value z, and a copy of the public key.

Algorithm 4 KeyGen()

$\sigma \leftarrow \{0,1\}^{512}$
$(pk, s) \leftarrow$ CPA.KeyGen$(\sigma[0{:}256])$
$z \leftarrow \mathrm{SHAKE256}(\sigma[256{:}512] \,\|\, \texttt{"_hankel_cca_reject"}, 32)$
return pk, $\;sk = (s, z, pk)$

Algorithm 5 Encaps(pk)

$M \leftarrow \{0,1\}^{256}$
$(\bar{K} \,\|\, c) \leftarrow \mathrm{SHAKE256}(M \,\|\, \mathrm{SHA256}(pk), 64)$
$ct \leftarrow$ CPA.Enc$(pk, M; c)$
return ct, $\;K = \mathrm{SHAKE256}(\bar{K} \,\|\, \mathrm{SHA256}(ct), 32)$

Algorithm 6 $\mathsf{Decaps}(sk, ct)$

Parse $sk = (s, z, pk)$
$M' \leftarrow \mathsf{CPA.Dec}(s, ct)$
$(\bar{K}' \,\|\, c') \leftarrow \mathrm{SHAKE256}(M' \,\|\, \mathrm{SHA256}(pk), 64)$
$ct' \leftarrow \mathsf{CPA.Enc}(pk, M'; c')$
$\hat{K} \leftarrow \bar{K}'$ if $ct = ct'$, else z ▷ constant-time comparison and selection
return $K = \mathrm{SHAKE256}(\hat{K} \,\|\, \mathrm{SHA256}(ct), 32)$

A ciphertext with the wrong length or a malformed header can never equal a re-encryption, so it is rejected implicitly like any other invalid ciphertext. Appendix B gives the byte-level formats.

5.3 Security of the scheme

The encryption scheme is IND-CPA secure if the decision problems of Section 4 are hard, by the usual argument for LWE-based encryption [15]. Decision $\mathsf{WH\text{-}LWE}_{640,(768),q,\mathrm{CBD}_2,\mathrm{CBD}_2}$ lets us replace the public key b by a uniform vector. The ciphertext then consists of $u = H_m(a)\, r + e_1$ and $v - \mathrm{PAM}(M) = H_W(b)\, r + e_2$, with a and b uniform and independent; since $H_W(b)$ uses only the first $n + W - 1 = 767$ of the 768 entries of b, this is an instance of decision $\mathsf{WH\text{-}LWE}_{640,(768,128),q,\mathrm{CBD}_2,\mathrm{CBD}_2}$ with secret r, and replacing it by uniform hides M. The order of the steps matters. In a real ciphertext, the matrix $H_W(b)$ of v is generated by $b = Hs + e$ and is therefore related to the matrix H of u; an attack that exploited this relation to eliminate r would distinguish b from uniform, and so would break the public-key instance.

IND-CCA security then follows from IND-CPA security and the correctness error, in the random oracle model [13] and in the quantum random oracle model [6, 14]; Section 7.3 shows $\delta \leq 2^{-134.1}$. Hashing $\mathrm{SHA256}(pk)$ into the coins binds each encapsulation to its public key, and hashing $\mathrm{SHA256}(ct)$ into the shared key binds the key to its ciphertext.

6 Parameter Selection

Table 2 lists the parameters and sizes. We explain each choice below and compare it with nearby alternatives. We carried out this comparison after fixing the parameters; it confirms the choice, and it turned up one alternative that deserves further study.

Parameter	Value
Dimension n	640
Window W / published coordinates $m = n + W$	128 / 768
Modulus q	$4096 = 2^{12}$
Secret and error distribution	CBD_2 (variance 1)
Public sequence length $L = 2n + W - 1$	1,407
Message	256 bits (4-PAM over W coordinates)
Public key	1,196 B
Ciphertext	1,356 B
Secret key	1,868 B

Table 2: Parameters of Windowed Hankel KEM. The public key consists of a 12-byte header, a 32-byte seed and 768 coefficients of 12 bits; the ciphertext of a 12-byte header and $768 + 128$ coefficients of 12 bits.

Modulus. A power-of-two modulus makes reduction a bitwise mask and lets uniform coefficients be read directly from masked 16-bit words of SHAKE256 output, without rejection sampling.

With $q = 2^{12}$, two coefficients also pack exactly into three bytes, so every byte string of the right length decodes to valid coefficients, and products can be accumulated in 16-bit integers (Section 8). The cost is that NTT-based multiplication is unavailable.

Encoding and window. Carrying two bits per coordinate halves the window needed for a 256-bit message, from 256 coordinates to 128. Every window coordinate appears three times on the wire, in b, u and v, so the smaller window saves 3×128 coefficients, or 576 bytes. In return, the decision threshold falls from $q/4$ to $q/8$, and the modulus must leave room for it.

Noise and modulus. Table 3 applies the Chernoff bound of Section 7.3 to several combinations, all with $n = 640$. With 4-PAM and CBD_2 noise, $q = 2^{12}$ is the smallest power of two that keeps the bound below 2^{-128}; at $q = 2^{11}$ the bound is only $2^{-28.5}$. Wider noise, CBD_3, would need $q = 2^{13}$ and one more bit per coefficient.

q	noise	bits per coordinate	threshold / σ	$\log_2$ DFR bound
2^{11}	CBD_1	2	14.30	−137.8
2^{11}	CBD_2	2	7.15	−28.5
2^{11}	CBD_2	1	14.31	−133.1
2^{12}	CBD_2	2	14.31	−134.1
2^{12}	CBD_3	2	9.54	−56.1
2^{13}	CBD_3	2	19.08	−234.6

Table 3: Chernoff bound on the decryption failure rate for $n = 640$ and a 256-bit message. The row with $q = 2^{12}$ and CBD_2 is the chosen parameter set.

Dimension. Table 4 shows how the cheapest primal and dual estimates move with n, and how the viable alternatives of Table 3 compare. At $n = 576$, the dual hybrid estimate would already be $2^{121.2}$ in the Core-SVP model, above Kyber-512's $2^{115.5}$; we take $n = 640$ to leave a margin for attacks that might exploit the structure.

			Core-SVP		MATZOV		
n	q	noise	primal	dual hybrid	primal	dual hybrid	PK + CT
512	2^{12}	CBD_2	108.3	105.4	134.1	129.6	2,168 B
576	2^{12}	CBD_2	125.3	121.2	150.3	145.1	2,360 B
640	$\mathbf{2^{12}}$	$\mathbf{CBD_2}$	**142.5**	**137.1**	**166.8**	**160.3**	**2,552 B**
704	2^{12}	CBD_2	159.7	153.3	183.4	176.1	2,744 B
640	2^{11}	CBD_1	141.6	134.3	165.9	157.7	2,344 B
640	2^{13}	CBD_3	136.9	133.7	161.7	157.4	2,760 B

Table 4: Estimated costs ($\log_2$) of the primal (uSVP) and dual hybrid attacks, and the combined size of public key and ciphertext including headers, assuming 11- and 13-bit coefficients pack as tightly as 12-bit ones. The chosen parameter set is in bold.

An alternative. The set $(n, q, \eta) = (640, 2^{11}, 1)$ is 208 bytes (8%) smaller and has a slightly lower failure bound, $2^{-137.8}$, for a dual hybrid estimate 2.8 bits lower. Its secret, though, is ternary with $\Pr[s_j = 0] = 1/2$, the regime in which combinatorial and hybrid attacks on small secrets improve, and the estimator's meet-in-the-middle optimizer did not converge for it in every cost model. We keep CBD_2 and leave this alternative to future work.

Higher categories. The same method yields parameters for NIST Categories 3 and 5, with $n = 936$ and $n = 1{,}208$, at the cost of one more bit per coefficient; Appendix D gives the derivation, and the C implementation covers both sets.

Comparison. Table 5 compares wire sizes with other LWE-based KEMs at the 128-bit level; for a like-for-like comparison, our figures exclude the 12-byte format headers. Ciphertexts are sent uncompressed. Rounding u and v to fewer bits, as ML-KEM does, would shrink them further, but would need a new failure analysis.

Scheme	Structure	Public key	Ciphertext	Total
ML-KEM-512 [20]	module-LWE	800 B	768 B	1,568 B
Windowed Hankel KEM (this work)	middle product	1,184 B	1,344 B	2,528 B
Titanium-CCA-Std128 [27]	MP-LWE	16,352 B	3,552 B	19,904 B
FrodoKEM-640 [8]	unstructured LWE	9,616 B	9,720 B	19,336 B

Table 5: Wire sizes of LWE-based KEMs targeting the 128-bit security level.

7 Security Analysis

7.1 Methodology

We treat each instance of Section 4 as unstructured LWE with secret dimension $n = 640$, modulus $q = 4096$, and secret and error of standard deviation 1, which amounts to assuming that the structure gives the attacker no advantage (Section 4.3 examines that assumption). Costs come from the lattice estimator [2] (commit `53da598`, August 2026) under two models for BKZ with block size β: the Core-SVP model [4], which charges $2^{0.292\beta}$ classically per call to an SVP oracle, and the more detailed model of MATZOV [17]. The same version of the estimator reproduces exactly the Kyber-512 figures in its documentation, and we computed every figure in this section in two SageMath installations (10.8 and 10.9) with identical results.

7.2 Generic attacks

Table 6 gives the estimates for the public key, which has $m = 768$ samples, alongside those for Kyber-512.

	Windowed Hankel KEM				Kyber-512	
	Core-SVP		MATZOV		Core-SVP	MATZOV
Attack	β	cost	β	cost	cost	cost
Primal (uSVP)	488	$2^{142.5}$	488	$2^{166.8}$	$2^{118.6}$	$2^{143.8}$
Primal (BDD)	488	$2^{143.2}$	471	$2^{163.1}$	$2^{119.4}$	$2^{140.2}$
Dual	507	$2^{148.0}$	509	$2^{173.7}$	$2^{123.8}$	$2^{149.9}$
Dual hybrid	468	$2^{137.1}$	459	$2^{160.3}$	$2^{115.5}$	$2^{139.7}$
Coded-BKW	—	—	—	$2^{207.3}$	—	$2^{178.8}$

Table 6: Lattice-estimator results for the public key ($n = 640$, $m = 768$, $q = 4096$, CBD_2 secret and error) and for Kyber-512, both treated as unstructured LWE. Costs are the estimator's total cost, including guessing in the hybrid attack. Coded-BKW is costed without lattice reduction, so it appears once per scheme.

The dual hybrid attack is the cheapest in both models. In the Core-SVP model it runs BKZ with $\beta = 468$, so lattice reduction alone costs $2^{0.292 \cdot 468} = 2^{136.7}$ classically and $2^{0.265 \cdot 468} = 2^{124.0}$ quantumly. The primal attack needs $\beta = 488$ in a lattice of dimension 1,194, at $2^{142.5}$ classically and $2^{129.3}$ quantumly; for Kyber-512, whose Category 1 claim rests on a classical Core-SVP cost of 2^{118} [9], the estimator gives a matching primal figure. Across the table, our instance costs at least 20 bits more than Kyber-512 under the same model, the smallest margin being 20.6 bits for the dual hybrid attack under MATZOV. This margin is the whole budget for an attack

that exploits the structure. The most natural such attack, through functionals derived from the cyclic embedding, reduces to the dual hybrid attack with a more expensive guess (Section 4.3 and Appendix C).

None of the remaining attack families comes close to the dual hybrid attack. The bounded support of the error lets each sample be written as a polynomial equation of degree 5 in s, which is the starting point of Arora–Ge attacks. Linearizing these equations would take about $\binom{n+5}{5} \approx 2^{39.7}$ samples, far more than the 768 available, and with 768 samples the estimator's Gröbner-basis model reaches a degree of regularity of 442, at a cost of about 2^{2100}. The primal hybrid attack is no cheaper than the plain primal attacks ($2^{143.2}$ and $2^{163.1}$ in the two models), and its meet-in-the-middle variant costs $2^{283.4}$. The meet-in-the-middle dual hybrid, designed for sparse or ternary secrets, costs $2^{748.0}$ against CBD_2.

Finally, none of the optimal attacks uses all the samples on offer: the primal attack uses 553, the dual attack about 620, and the dual hybrid attack 640, out of 768. The 896 samples that a ciphertext provides in the secret r therefore do not lower the estimates either.

7.3 Decryption failure rate

Theorem 1. The decryption failure rate satisfies $\delta \leq 2^{-134.1}$.

Proof. For each window coordinate w, $e_{\mathrm{dec},w}$ is a sum of 1,281 independent terms: 1,280 products XY of independent CBD_2 variables, namely $r_j e_{w+j}$ and $-s_k e_{1,w+k}$ (the distribution of XY is symmetric), and the CBD_2 variable $e_{2,w}$. Each term has mean 0 and variance 1, so $\sigma = \sqrt{1281} \approx 35.79$ and the threshold 512 lies 14.31σ from the mean. The Chernoff bound with the exact moment-generating functions (Appendix A) gives $\Pr[e_{\mathrm{dec},w} \geq 512] \leq 2^{-142.1}$, so by symmetry $e_{\mathrm{dec},w}$ leaves $[-512, 511]$ with probability at most $2^{-141.1}$. A union bound over the $W = 2^7$ coordinates gives $\delta \leq 2^{-134.1}$. □

As usual, the bound is an average over key pairs. FIPS 203 lists $2^{-138.8}$ for ML-KEM-512 [20, Table 1]. Experiments cannot confirm a rate this small, but they agree with the bound: over 500 end-to-end exchanges (64,000 decoded coordinates), the largest $|e_{\mathrm{dec}}|$ was 131, and its mean and standard deviation, 28.4 and 21.4, match the values 28.5 and 21.6 predicted by a normal approximation.

Failure rate matters beyond correctness, because attacks that collect decryption failures can erode the security of LWE-based schemes whose failure rate is relatively high [11]. The 128 window coordinates share r, s and e_1, so their errors are correlated. The union bound in Theorem 1 remains valid, and is if anything conservative, but a failure, should one occur, reveals more about the secret than an independent-coordinate model would suggest. At $\delta \leq 2^{-134.1}$ this is moot in practice: an attacker who submits 2^{64} honestly generated ciphertexts expects fewer than 2^{-70} failures.

8 Implementation

We implemented the KEM twice: a Python reference and a C implementation of the complete scheme. The C code is bit-exact with the reference. It reproduces all fifteen known-answer test vectors, which come from a custom SHAKE-based generator rather than the NIST DRBG, and matches the reference on random inputs, on every branch of its seed handling and on malformed ciphertexts.

8.1 Arithmetic

All three Hankel products have the form $y_i = \sum_{j<n} a_{i+j} x_j \ (+\, e_i) \bmod q$. Because $q = 2^{12}$ divides 2^{16}, they can be computed modulo 2^{16}, using the natural wraparound of 16-bit integers, and

reduced modulo q by a single mask at the end; this is exact for any 16-bit inputs, including negative secret coefficients in two's complement. On ARMv8, the kernel computes 64 outputs at a time in eight 8-lane accumulators. For each secret coefficient x_j it adds $x_j \cdot (a_{i+j}, \ldots, a_{i+j+63})$ to the accumulators with `vmlaq_laneq_u16`, so a 768-output product takes $768 \cdot 640/8 = 61{,}440$ multiply-accumulate instructions, and the secret only ever enters as a multiplication operand. A portable scalar kernel is kept and tested alongside.

8.2 Hashing

SHAKE256 uses a Keccak-$f[1600]$ permutation with fixed loop counts and no table lookups. SHA-256 only ever hashes public data, the public key and the ciphertext; it uses the ARMv8 SHA-256 instructions when available, and is checked against a portable implementation.

8.3 Constant-time execution

No secret value selects a branch or a memory address. The all-zero-secret fix-up of Section 5, the comparison $ct = ct'$ and the selection of $\hat{K}$ in Algorithm 6 use masks behind a compiler value barrier, so that the compiler cannot turn them back into branches. Every step that handles secrets is compiled as a separate function, which makes two kinds of check possible.

The first is a static audit of the compiled ARM64 code: a flow-sensitive dataflow analysis tracks which registers, stack slots and condition flags may depend on secret data, and reports any conditional branch on such a value, any division and any unexpected call. Across the nine secret-handling functions (1,274 instructions), it finds 23 conditional branches, all on public values, and no divisions or calls. It does not check memory addressing, but every index in these functions is a loop counter, which we verified by inspection.

The second is a set of fixed-versus-random Welch t-tests with about 4,000 calls per class, varying only the secret input: the seed for key generation, the message entropy for encapsulation, and, for decapsulation of a fixed ciphertext, the secret key, so that one class accepts and the other rejects. All three give $|t| < 4.5$. Repeating the decapsulation test with 20,000 calls per class, including a control that compares a class with itself, gave $|t| \leq 2.3$ and medians within 10 ns: decapsulation takes the same time whether it accepts or rejects. This evidence covers timing only, on a single core with a single compiler; we have not studied power or electromagnetic leakage.

8.4 Performance

On an Apple M3 Max, with Apple clang 17 at `-O3`, the median latencies over 5,000 calls with fresh inputs are 14 μs for key generation, 17 μs for encapsulation and 20 μs for decapsulation, or 51 μs for a full handshake. An encapsulation spends about 8 μs in the two Hankel products and about 6 μs in SHAKE256, mostly expanding the public sequence. Faster Keccak would help, and so would subquadratic multiplication; without an NTT, the products stay quadratic in n. We do not compare timings with other schemes, which we did not measure under the same conditions.

The implementation is available at `https://github.com/Cryptozoa/windowed-hankel-kem`, together with the scripts that produce every figure in this paper and their raw outputs.

9 Limitations and Open Questions

The central limitation is the one we accepted by design: Windowed Hankel LWE has no reduction, and its hardness is an assumption. The estimates of Section 7 hold only if the structure gives an attacker no advantage, and the roughly 20-bit margin over Kyber-512 is all that a structural attack would need to overcome. We have checked the published structure-exploiting attacks and the dual-type functionals that arise from the embedding into $\mathbb{Z}_q[X]/(X^L - 1)$, and found no advantage, but this does not rule out other attacks. Several directions remain open: dual attacks

that look for functionals merely *biased* toward a low-dimensional function of the secret, rather than exactly isolating one; primal (bounded-distance decoding) attacks that use the subrings $\mathbb{Z}_q[X]/(g)$ while treating the 639 withheld coefficients as unknowns; algebraic decoding, since a noise-free Hankel system is a Padé-type problem that the extended Euclidean algorithm solves in $O(n^2)$ operations and a version tolerating small noise would be a new attack; and attacks on the joint distribution of a public key and its ciphertexts. We also have no search-to-decision reduction, and the usual arguments favor a prime modulus. The setting has received little study, and the fate of Compact-LWE [7] counsels caution; we offer the scheme as a problem for cryptanalysis, not for use, until it has been analyzed independently.

Two further limitations are practical. Without an NTT, multiplication costs $O(nm)$ operations, about 4.9×10^5 multiply-accumulates at $n = 640$. That is fast enough here, but it scales less well than ring-based schemes at higher security levels. Our constant-time evidence covers only timing, on one platform (Section 8).

10 Conclusion

Publishing a single long sequence and keeping the secret short brings an LWE-based KEM to 2.5 KB per exchange without reducing products modulo any polynomial, at the cost of the MP-LWE reductions. If the structure gives an attacker nothing, generic attacks cost at least 20 bits more than against Kyber-512. The published structural attacks do not transfer, and the most natural new one, through the hidden cyclic embedding, reduces to a generic attack with a more expensive guess. The structure has yet to face sustained independent cryptanalysis, and we hope the precise problem statement given here makes that analysis easier to undertake.

Acknowledgments

We thank two colleagues for detailed comments on an earlier draft, which shaped the analysis of Sections 4.2 and 4.3. Portions of the analysis, the implementation and the manuscript were prepared with the assistance of Claude (Anthropic), an AI model. The author takes full responsibility for the content.

References

[1] Gorjan Alagic, Jacob Alperin-Sheriff, Daniel Apon, David Cooper, Quynh Dang, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, and Daniel Smith-Tone. Status report on the first round of the NIST post-quantum cryptography standardization process. Technical Report NISTIR 8240, National Institute of Standards and Technology, 2019.

[2] Martin R. Albrecht et al. Security estimates for lattice problems (lattice-estimator). Software.

[3] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9(3), 2015.

[4] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange – a new hope. In *25th USENIX Security Symposium*, 2016.

[5] Shi Bai, Dipayan Das, Ryo Hiromasa, Miruna Roşca, Amin Sakzad, Damien Stehlé, Ron Steinfeld, and Zhenfei Zhang. MPSign: A signature from small-secret middle-product learning with errors. In *Public-Key Cryptography – PKC 2020*, Lecture Notes in Computer Science. Springer, 2020. Cryptology ePrint Archive, Paper 2020/198.

[6] Nina Bindel, Mike Hamburg, Kathrin Hövelmanns, Andreas Hülsing, and Edoardo Persichetti. Tighter proofs of CCA security in the quantum random oracle model. In *Theory of Cryptography – TCC 2019*, Lecture Notes in Computer Science. Springer, 2019. Cryptology ePrint Archive, Paper 2019/590.

[7] Jonathan Bootle, Mehdi Tibouchi, and Keita Xagawa. Cryptanalysis of Compact-LWE. In *Topics in Cryptology – CT-RSA 2018*, volume 10808 of *Lecture Notes in Computer Science*, pages 80–97. Springer, 2018.

[8] Joppe Bos, Craig Costello, Léo Ducas, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Ananth Raghunathan, and Douglas Stebila. Frodo: Take off the ring! practical, quantum-secure key exchange from LWE. In *ACM Conference on Computer and Communications Security (CCS) 2016*. ACM, 2016.

[9] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM. In *IEEE European Symposium on Security and Privacy (EuroS&P) 2018*. IEEE, 2018.

[10] Yihang Cheng, Xiangzheng Zhao, Hengyi Luo, Yanbin Pan, and Siwei Sun. Tighter nonuniform trace reductions from Ring-LWE to MP-LWE. Cryptology ePrint Archive, Paper 2026/2151, 2026.

[11] Jan-Pieter D'Anvers, Frederik Vercauteren, and Ingrid Verbauwhede. On the impact of decryption failures on the security of LWE/LWR based schemes. In *Public-Key Cryptography – PKC 2019*, Lecture Notes in Computer Science. Springer, 2019. Cryptology ePrint Archive, Paper 2018/1089.

[12] Yara Elias, Kristin E. Lauter, Ekin Ozman, and Katherine E. Stange. Provably weak instances of Ring-LWE. In *Advances in Cryptology – CRYPTO 2015*, Lecture Notes in Computer Science. Springer, 2015. arXiv:1502.03708.

[13] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In *Theory of Cryptography – TCC 2017*, volume 10677 of *Lecture Notes in Computer Science*. Springer, 2017.

[14] Haodong Jiang, Zhenfeng Zhang, Long Chen, Hong Wang, and Zhi Ma. IND-CCA-secure key encapsulation mechanism in the quantum random oracle model, revisited. In *Advances in Cryptology – CRYPTO 2018*, Lecture Notes in Computer Science. Springer, 2018. Cryptology ePrint Archive, Paper 2017/1096.

[15] Richard Lindner and Chris Peikert. Better key sizes (and attacks) for LWE-based encryption. In *Topics in Cryptology – CT-RSA 2011*, volume 6558 of *Lecture Notes in Computer Science*, pages 319–339. Springer, 2011.

[16] Alex Lombardi, Vinod Vaikuntanathan, and Thuy-Duong Vuong. Lattice trapdoors and IBE from middle-product LWE. In *Theory of Cryptography – TCC 2019*, Lecture Notes in Computer Science. Springer, 2019. Cryptology ePrint Archive, Paper 2019/1067.

[17] MATZOV. Report on the security of LWE: Improved dual lattice attack. Technical report, Zenodo, 2022.

[18] National Institute of Standards and Technology. Secure hash standard (SHS). Federal Information Processing Standards Publication 180-4, 2015.

[19] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Federal Information Processing Standards Publication 202, 2015.

[20] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Federal Information Processing Standards Publication 203, 2024.

[21] Rahinatou Yuh Njah Nchiwo and Alice Pellet-Mary. Improved reduction from RLWE to MP-LWE. In *Public-Key Cryptography – PKC 2026*, Lecture Notes in Computer Science. Springer, 2026. Cryptology ePrint Archive, Paper 2026/342.

[22] Chris Peikert. A decade of lattice cryptography. *Foundations and Trends in Theoretical Computer Science*, 10(4), 2016.

[23] Chris Peikert and Zachary Pepin. Algebraically structured LWE, revisited. In *Theory of Cryptography – TCC 2019*, Lecture Notes in Computer Science. Springer, 2019. Journal of Cryptology, 2024. Cryptology ePrint Archive, Paper 2019/878.

[24] Alice Pellet-Mary and Wenwen Xia. A unified reduction from RLWE to MP-LWE. In *Advances in Cryptology – ASIACRYPT 2026*, 2026. Cryptology ePrint Archive, Paper 2026/2026.

[25] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6), 2009.

[26] Miruna Roşca, Amin Sakzad, Ron Steinfeld, and Damien Stehlé. Middle-product learning with errors. In *Advances in Cryptology – CRYPTO 2017*, Lecture Notes in Computer Science. Springer, 2017. Cryptology ePrint Archive, Paper 2017/628.

[27] Ron Steinfeld, Amin Sakzad, and Raymond K. Zhao. Titanium: Proposal for a NIST post-quantum public-key encryption and KEM standard. Submission to the NIST Post-Quantum Cryptography Standardization Process, Round 1, 2017.

[28] Ron Steinfeld, Amin Sakzad, and Raymond K. Zhao. Practical MP-LWE-based encryption balancing security-risk versus efficiency. *Designs, Codes and Cryptography*, 2019. Cryptology ePrint Archive, Paper 2019/1179.

[29] Masayuki Tezuka and Keisuke Tanaka. A survey on middle-product learning with errors cryptography. Book chapter, Springer Singapore, 2025.

[30] Shaojun Yang and Xinyi Huang. Universal product learning with errors: A new variant of LWE for lattice-based cryptography. *Theoretical Computer Science*, 915:90–100, 2022.

A The Chernoff Bound

Let $Y = XX'$ for independent $X, X' \sim \mathrm{CBD}_2$. Its distribution is

$$\Pr[Y=0] = \tfrac{156}{256}, \quad \Pr[Y=\pm 1] = \tfrac{32}{256}, \quad \Pr[Y=\pm 2] = \tfrac{16}{256}, \quad \Pr[Y=\pm 4] = \tfrac{2}{256},$$

with mean 0 and variance 1. Write $e_{\mathrm{dec},w} = \sum_{i=1}^{1280} Y_i + Z$ with $Z \sim \mathrm{CBD}_2$. Markov's inequality applied to $e^{te_{\mathrm{dec},w}}$ gives, for every $t > 0$,

$$\Pr[e_{\mathrm{dec},w} \geq 512] \leq \exp\bigl(1280 \ln M_Y(t) + \ln M_Z(t) - 512\, t\bigr),$$

where $M_Y(t) = \mathbb{E}[e^{tY}]$ is computed exactly from the distribution above and $M_Z(t) = \cosh^4(t/2)$. The exponent is convex in t and minimized at $t^* \approx 0.3715$, where the bound equals $2^{-142.07}$. The same computation, with 1280 replaced by $2n$, the threshold by $q/2^{b+1}$ for b bits per coordinate and the union bound taken over $256/b$ coordinates, produces every entry of Table 3.

B Wire Formats and Derivation Tags

All multi-byte integers are little-endian, and coefficients are packed in pairs (x_0, x_1) into the three bytes $(x_0 \bmod 2^8,\ \lfloor x_0/2^8 \rfloor + 16(x_1 \bmod 2^4),\ \lfloor x_1/2^4 \rfloor)$. Table 7 gives the layouts.

Object	Bytes	Layout
Public key	1,196	`"HNPK"`, then n, W, q, 0 as 16-bit integers (12 bytes); $seed_a$ (32); b packed (1,152)
Ciphertext	1,356	`"HNCT"`, then n, W, q, 0 (12 bytes); u packed (1,152); v packed (192)
Secret key	1,868	s as signed bytes (640); z (32); public key (1,196)

Table 7: Byte layouts.

Each SHAKE256 call is domain-separated by an ASCII tag appended to its input, listed in Table 8. The first four belong to the algorithms of Section 5. The last two exist only for deterministic testing, and the known-answer tests use them: the implementations accept a key-generation seed of any length, expanding one shorter than 64 bytes first, and can derive the encapsulated message from caller-supplied entropy.

Tag	Use
`_hankel_cpa_keygen_seed`	$(seed_a \Vert seed_e)$ from σ[0:256] (Algorithm 1)
`_hankel_noise`	s and e from $seed_e$ (Algorithm 1)
`_hankel_encaps_coins`	r, e_1 and e_2 from the coins c (Algorithm 2)
`_hankel_cca_reject`	rejection value z from σ[256:512] (Algorithm 4)
`_hankel_cca_keygen_seed`	testing: expands a key-generation seed shorter than 64 bytes
`_hankel_encaps_entropy`	testing: $M = \mathrm{SHAKE256}(\rho \Vert \mathrm{tag}, 32)$ from entropy ρ

Table 8: Domain-separation tags.

C Short Functionals from the Cyclic Embedding

This appendix gives the computations behind the embedding argument of Section 4.3.

The lattice. Let g be a monic factor of $X^L - 1$ over $\mathbb{Z}_q$ of degree k, and let $R_g \in \mathbb{Z}_q^{k \times n}$ have as its column j the coefficient vector of $X^{n-1-j} \bmod g$, so that $R_g s$ is the coefficient vector of $\tilde{s} \bmod g$. The last k columns of R_g are unit vectors, so the condition $H_m(a)^\top \varphi = R_g^\top c$ determines c from k coordinates of $H_m(a)^\top \varphi$ and reduces to $n-k$ linear conditions $B_g \varphi \equiv 0 \pmod q$, with $B_g \in \mathbb{Z}_q^{(n-k)\times m}$. The functionals form the lattice $\Lambda_g = \{\varphi \in \mathbb{Z}^m : B_g \varphi \equiv 0 \pmod q\}$; for $k = 0$ it is the lattice of the plain dual attack. If B_g has full rank modulo 2, it maps $\mathbb{Z}^m$ onto $\mathbb{Z}_q^{n-k}$ and $\det \Lambda_g = q^{n-k}$, exactly as for a uniform matrix, and as for the dual-hybrid lattice obtained by guessing k coordinates of s. We estimate the shortest vector by the Gaussian heuristic $\mathrm{GH}(\Lambda_g) = \Gamma(m/2+1)^{1/m} \det(\Lambda_g)^{1/m}/\sqrt{\pi}$. The entropy of $\tilde{s} \bmod g$ is estimated with a Gaussian of covariance $R_g R_g^\top$, using centered representatives, and capped at 12 bits per coefficient; for the cubic and sextic factors the entries of R_g are large modulo q, the cap applies, and $\tilde{s} \bmod g$ is close to uniform.

Full size. Table 9 covers $n = 640$, $m = 768$ and five random sequences a. In every case B_g has full rank modulo 2, so the structured lattices are exactly as dense as the corresponding dual-hybrid lattices, and the comparison reduces to the guess. The Gaussian heuristic also places the shortest functionals above q: $\varphi^\top e$ then has standard deviation larger than the modulus, so a single functional reveals almost nothing, and any use must combine many of them, as the dual attack does. We do not convert these figures into an attack cost; that is what the estimates of

Section 7 provide for the dual hybrid attack, which, by the argument above, these functionals do not beat.

		full rank			bits to guess	
Factor g	k	mod 2	$\log_2 \det$	GH	$\tilde{s}$ mod g	k coords of s
none (plain dual)	0	5/5	7,680	6,902	—	—
$X-1$	1	5/5	7,668	6,827	6.7	2.0
Φ_3	2	5/5	7,656	6,754	12.6	4.1
factors of Φ_7 (each)	3	5/5	7,644	6,681	36.0	6.1
factors of Φ_{21} (each)	6	5/5	7,608	6,467	71.0	12.2

Table 9: Lattices of functionals at full size ($n = 640$, $m = 768$, $q = 4096$, five random sequences).

Reduced sizes. To test whether Hankel structure produces unusually short functionals, we computed exact shortest vectors at reduced sizes, with fpylll enumeration after BKZ preprocessing, both for B_g derived from random Hankel matrices and for uniform matrices of the same shape. We chose $(n, W) = (40, 5)$, $(48, 10)$ and $(56, 15)$, giving lattice dimensions 45, 58 and 71, so that 21 divides L and all six factors above divide $X^L - 1$. Instances in which B_g was not of full rank modulo 2 were resampled; this happened once for a Hankel matrix and six times for a uniform one, all at the smallest size. Table 10 summarizes the ratio λ_1/GH over the seven targets of Table 9; the evidence directory gives the figures per target.

				Hankel		uniform	
n	W	m	instances per target	means	smallest	means	smallest
40	5	45	20	0.990–1.016	0.888	0.997–1.016	0.903
48	10	58	20	0.996–1.010	0.886	0.998–1.008	0.945
56	15	71	5	0.991–1.010	0.960	0.995–1.019	0.958

Table 10: Exact shortest vectors of the lattices of functionals, as a ratio to the Gaussian heuristic: range of the per-target means over the seven targets (including the plain dual lattice), and the smallest ratio over all instances of that size.

The per-target means agree with those of uniform matrices to within about 2%. The smallest Hankel ratio, 0.886, lies within the spread of the individual instances: at the smallest size, Hankel and uniform matrices reach 0.888 and 0.903. We found no sign that Hankel structure shortens the functionals.

The scripts and their outputs are in the accompanying evidence directory.

D Higher Security Categories

This appendix derives candidate parameters for NIST Categories 3 and 5 by the method of Section 6. We size them as the Category 1 set is sized: the cheapest estimated attack should exceed that on the corresponding Kyber parameter set by at least 20 bits in both cost models, and the failure rate should be no larger than ML-KEM's. Under the estimator and models of Section 7, the cheapest attacks on Kyber-768 and Kyber-1024 cost $2^{174.3}$ and $2^{241.8}$ (Core-SVP) and $2^{196.4}$ and $2^{262.3}$ (MATZOV); FIPS 203 lists failure rates of $2^{-164.8}$ and $2^{-174.8}$ for ML-KEM-768 and ML-KEM-1024 [20, Table 1]. The message stays at 256 bits.

The Category 1 recipe does not scale. The decryption error has variance $2n + 1$, so the failure rate grows with n. With $q = 2^{12}$, CBD_2 and 4-PAM, the Chernoff bound of Theorem 1

exceeds $2^{-164.8}$ for every $n \geq 640$; at $n = 640$ it is already $2^{-134.1}$. Four modifications reach the targets:

A. $q = 2^{13}$, keeping CBD_2 and 4-PAM, at the cost of one more bit per coefficient;

B. one bit per coordinate ($W = 256$), keeping $q = 2^{12}$ and CBD_2;

C. CBD_1 secret and noise, keeping $q = 2^{12}$ and 4-PAM;

D. $q = 2^{13}$ with CBD_3 and one bit per coordinate.

Table 11 gives the smallest n that meets the targets in each family. Family C is the smallest, but its secret is the sparse ternary one set aside in Section 6. Families A and B are within 102 bytes of each other at both levels. We choose family A because it keeps the Category 1 scheme unchanged except for q, and because both of its lengths L are prime with $X - 1$ as the only small factor of $X^L - 1$; for family B at Category 5, $L = 2{,}511$ gives factors of degrees 2, 5, 6, 10 and 18.

	Category 3		Category 5	
Family	n	PK + CT	n	PK + CT
A: $q = 2^{13}$, CBD_2, 4-PAM	936	3,722 B	1,208	4,606 B
B: $q = 2^{12}$, CBD_2, 1 bit	872	3,824 B	1,128	4,592 B
C: $q = 2^{12}$, CBD_1, 4-PAM	960	3,512 B	1,280	4,472 B
D: $q = 2^{13}$, CBD_3, 1 bit	896	4,216 B	1,152	5,048 B

Table 11: Smallest n meeting the Category 3 and 5 targets, by family, with the combined size of public key and ciphertext including headers.

Proposed sets. Table 12 lists the family A sets. In both, the dual hybrid attack remains the cheapest, the margin over Kyber is at least 21 bits in both models, and the failure rate is far below the target, which leaves room for compressing ciphertexts later. Merely matching Kyber, without the 20-bit margin, would require $n = 848$ and $n = 1{,}120$, at 3,436 and 4,320 bytes.

	Category 3	Category 5
n, W, $m = n + W$	936, 128, 1,064	1,208, 128, 1,336
q, distribution, encoding	2^{13}, CBD_2, 4-PAM	2^{13}, CBD_2, 4-PAM
Dual hybrid, Core-SVP (β)	$2^{196.8}$ (674)	$2^{263.5}$ (902)
Dual hybrid, MATZOV (β)	$2^{218.2}$ (665)	$2^{283.7}$ (899)
Primal (uSVP), Core-SVP	$2^{205.9}$	$2^{277.7}$
Failure rate (Chernoff bound)	$2^{-370.1}$	$2^{-291.9}$
Public key / ciphertext	1,773 B / 1,949 B	2,215 B / 2,391 B
PK + CT without headers (ML-KEM)	3,698 B (2,272 B)	4,582 B (3,136 B)
$L = 2n + W - 1$	1,999 (prime)	2,543 (prime)

Table 12: Proposed parameters for Categories 3 and 5 (lattice estimator, commit `53da598`).

Ring structure. The dimension n of Windowed Hankel LWE can be any integer, and the factorization of $X^L - 1$ varies widely with it. For example, $n = 960$ in family A gives $L = 2{,}047 = 2^{11} - 1$, for which $X^L - 1$ has 186 irreducible factors of degree 11. The proposed sets happen to fall on the favorable side: $L = 1{,}999$ and $L = 2{,}543$ are prime, and 2 has order 333 and 1,271 modulo them, so $X - 1$ is the only factor of $X^L - 1$ of degree below 333. Of the low-degree factors examined in Appendix C, only $X - 1$ remains. Repeating that check at the new sizes gives the same result: for five random sequences each, the lattices of functionals for the plain dual attack and for $X - 1$ have full rank modulo 2, hence the determinants of the corresponding dual-hybrid lattices, while isolating $s(1)$ requires guessing 7.0 and 7.2 bits against 2.0 bits for a coordinate of s. Choosing L prime with 2 of large order is a free design rule for any further parameter set.

Implementation. Both sets are built from the same C code as the Category 1 set (Section 8), compiled once per set. Since 2^{13} divides 2^{16}, accumulating in 16-bit lanes stays exact, and the only changes are the packing, at 13 bits per coefficient and least significant bit first, and the 4-PAM scaling by $q/4$. To keep the sets apart, every derivation tag of Appendix B is followed by a domain string, `_wh936` or `_wh1208`; the tags of the Category 1 set are unchanged. Each set has 15 known-answer vectors, generated by a parameterized Python reference, and the C code reproduces them exactly. It passes the same static branch audit and fixed-versus-random timing tests as the Category 1 code, with $|t| < 4.5$ in all six tests. On the machine of Section 8, median latencies over 5,000 calls are 30, 31 and 37 μs for key generation, encapsulation and decapsulation at Category 3, and 40, 48 and 48 μs at Category 5, or about 100 and 135 μs for a full handshake.

What scaling does not change. These sets rest on the same assumption as the Category 1 set, and a larger n adds no evidence for it. The open questions of Section 9 apply unchanged, and they matter more, not less, at the levels intended for long-term security.